คู่มือทบทวนและเฉลยข้อสอบฉบับสมบูรณ์: ครอบคลุมทั้ง LECTURE และ LAB

วิชา Digital Circuit Design with Verilog HDL (Week 2 ถึง Week 8)

บทนำ

เอกสารฉบับนี้จัดทำขึ้นเพื่อเป็น คู่มือการเรียนการสอนและเตรียมสอบฉบับสมบูรณ์ (Master Comprehensive Study Guide)
โดยรวบรวมและวิเคราะห์เนื้อหาเชิงทฤษฎีจาก สไลด์บรรยาย (Lecture) ควบคู่ไปกับเฉลยและคำอธิบายเชิงปฏิบัติจาก ใบงานปฏิบัติการ (Lab Sheet) ตั้งแต่ Week 2 ถึง Week 8 ไว้อย่างครบถ้วน 100%

ทุกหัวข้อมีการระบุแหล่งที่มาอย่างชัดเจนว่ามาจาก LECTURE หรือ LAB (ชื่อไฟล์, สไลด์, หน้า, ข้อ) โดยไม่มีการนำความรู้ภายนอกมาเจือปน
เพื่อให้ผู้เรียนใช้เป็นแหล่งอ้างอิงที่แม่นยำที่สุดในการเตรียมสอบ

[สารบัญ] สารบัญ

1. [Week 2: Quartus II Design Flow & DE0 Setup (Lec & Lab 2)](#week-2)
2. [Week 3: Foundations & Verilog Language Basics (Lec & Lab 3)](#week-3)
3. [Week 4: Combinational Logic Design (Lec & Lab 4)](#week-4)
4. [Week 5: Sequential Logic Design (Lec & Lab 5)](#week-5)
5. [Week 6: Counters & Shift Registers (Lec & Lab 6)](#week-6)
6. [Week 7: Finite State Machine (FSM) Design (Lec & Lab 7)](#week-7)
7. [Week 8: Testbench Design & Functional Verification (Lec & Lab 8)](#week-8)
8. [ตารางสรุปภาพรวมความเชื่อมโยงตลอดหลักสูตร Week 2-8](#summary-table)

[WEEK] Week 2: Quartus II Design Flow & DE0 Setup

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week2_software installation.pdf](lec/Week2_software%20installation.pdf) (Slide 1-25)

1. วัตถุประสงค์และภาพรวม (Slide 2-3)

- เข้าใจสถาปัตยกรรมของชิป FPGA และบอร์ดทดลอง Altera DE0
- เข้าใจลำดับขั้นตอน FPGA Design Flow ในโปรแกรม Quartus II
- รู้วิธีการคอมไพล์ สังเคราะห์วงจร กำหนดขาพิน และดาวน์โหลดไฟล์คอนฟิกูเรชันลงสู่ชิปจริง

2. สเปกฮาร์ดแวร์ชิป Altera Cyclone III บนบอร์ด DE0 (Slide 5)

- รหัสชิปเป้าหมาย: EP3C16F484C6

- ตระกูล (Family): Cyclone III

- จำนวน Logic Elements (LEs): 15,408 LEs

- บล็อกหน่วยความจำฝังในชิป: 56 บล็อก M9K (ความจุรวม 504 Kbits RAM)

- วงจรคูณสัญญาณฮาร์ดแวร์: 56 Embedded Multipliers (18x18-bit)

- วงจรจัดการสัญญาณนาฬิกา: 4 Phase-Locked Loops (PLLs)

- จำนวนขา I/O สูงสุด: 346 User I/O Pins

- ตัวถังและความเร็ว: 484-pin FineLine BGA (FBGA), Speed Grade 6

- เหตุผลที่ต้องใช้ Quartus II v13.0sp1 (Slide 10):

- Quartus II Web Edition v13.0sp1 เป็นเวอร์ชันฟรีสุดท้ายที่ยังคงรองรับชิปตระกูล Cyclone III ก่อนที่ Altera จะตัดการรองรับออกตั้งแต่เวอร์ชัน 13.1 เป็นต้นไป

3. ลำดับขั้นตอน FPGA Design Flow (Slide 7-15)

1. Design Entry: การป้อนแบบวงจรผ่านภาษา Verilog HDL (.v), VHDL หรือ Schematic

2. Analysis & Synthesis: ตรวจสอบไวยากรณ์ แปลงโค้ดภาษาฮาร์ดแวร์ให้อยู่ในรูปสมการตรรกะระดับเกต (Logic Synthesis)

3. Fitter (Place & Route): จัดสรรวงจรลงใน Logic Elements (LEs) จริง และเดินสายสัญญาณเชื่อมต่อ (Routing) ตามข้อจำกัดของ Pin Assignment

4. TimeQuest Timing Analysis: คำนวณความเร็วสูงสุด ($F_{\max}$) และตรวจสอบเงื่อนไขเวลา Setup Time / Hold Time ของ Flip-Flop

5. Assembler: สร้างไฟล์ไบนารีสำหรับโปรแกรมชิป:
- .sof (SRAM Object File): ใช้โปรแกรมลง SRAM ชั่วคราวผ่านโหมด JTAG (ข้อมูลจะหายเมื่อดับไฟ)
- .pof (Programmer Object File): ใช้โปรแกรมลงชิปแฟลชคอนฟิกูเรชัน (EPCS) ผ่านโหมด Active Serial (AS)

6. Programmer: สื่อสารผ่านสาย USB-Blaster เพื่อดาวน์โหลดไฟล์เข้าสู่ชิป FPGA

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 2

แหล่งอ้างอิง: [Practice Lab Sheet 2.pdf](lab/Practice%20Lab%20Sheet%202.pdf) (หัวข้อที่ 1-4)

1. โจทย์และวัตถุประสงค์ของ Lab
- สร้างโปรเจกต์ใหม่ใน Quartus II โดยกำหนดชิปให้ถูกต้องเป็น EP3C16F484C6
- เขียนโมดูล Verilog พื้นฐาน กำหนดขาพินผ่าน Pin Planner และทดสอบการทำงานบนบอร์ดจริง

2. ข้อมูลพอร์ตฮาร์ดแวร์บนบอร์ด DE0 (Practice Lab Sheet 2 หัวข้อที่ 2)
- สวิตช์เลื่อน (Slide Switches): SW[9:0] เป็น Active-High (เลื่อนขึ้น = 1, เลื่อนลง = 0)
- ปุ่มกด (Push Buttons): KEY[2:0] เป็น Active-Low (ไม่กด = 1, กด = 0) มีวงจร Pull-up ในตัว
- หลอดไฟ LED สีเขียว: LEDG[9:0] เป็น Active-High (1 = ไฟติด, 0 = ไฟดับ)
- จอแสดงผล 7 ส่วน: HEX0 ถึง HEX3 เป็นแบบ Common Anode (Active-Low) สัญญาณ 0 สว่าง, 1 ดับ
- สัญญาณนาฬิกาหลัก: CLOCK_50 ความถี่ 50 MHz (คาบเวลา $T = 20\text{ ns}$) ต่อเข้าขาพิน PIN_G21

[WEEK] Week 3: Foundations & Verilog Language Basics

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week3_foundations and language basics.pdf](lec/Week3_foundations%20and%20language%20basics.pdf) (Slide 1-25)

1. วัตถุประสงค์และภาพรวม (Slide 2)
- เข้าใจโครงสร้างโมดูลพื้นฐานของ Verilog HDL
- เข้าใจความแตกต่างของ Data Types: wire และ reg
- เข้าใจระบบตรรกะ 4 ระดับ (4-Value Logic) และรูปแบบตัวเลข (Number Literals)
- การใช้งาน Gate Primitives, Continuous Assignment (assign) และตัวดำเนินการ (Operators)

2. โครงสร้างโมดูลและการเชื่อมต่อพอร์ต (Slide 6-8)
- โมดูล (Module): หน่วยโครงสร้างพื้นฐานที่สุด ประกอบด้วยชื่อโมดูล, รายการพอร์ตอินพุต/เอาต์พุต และลอจิกภายใน

- รูปแบบการเชื่อมโยงพอร์ต (Port Mapping):
- Positional Mapping: ต่อตามลำดับพอร์ต เช่น adder u1 (a, b, sum); เสี่ยงต่อการต่อสายผิดหากโมดูลย่อยเปลี่ยนลำดับ

- Named Mapping (แนะนำ): ต่อตามชื่อพอร์ต เช่น adder u1 (.a(a), .b(b), .sum(sum)); ปลอดภัยสูงสุด ลำดับสลับกันได้

3. Data Types และ Logic Values (Slide 12-15)
- wire (Net): ใช้แทนสายสัญญาณเชื่อมต่อ ไม่สามารถเก็บค่าได้ ต้องถูกขับอย่างต่อเนื่องด้วย assign หรือ submodule output

- reg (Variable): ตัวแปรที่เก็บค่าได้ใน Procedural blocks (always, initial) ไม่ได้แปลว่าต้องเป็น Flip-Flop เสมอไป หากใช้ใน always @() ที่มีเงื่อนไขครบถ้วน จะสังเคราะห์ได้วงจร Combinational

- ระบบตรรกะ 4 ระดับ (4-Value Logic):
- 0: ลอจิกลอยต่ำ (Logic 0 / False / Ground)
- 1: ลอจิกลอยสูง (Logic 1 / True / VCC)
- X: ไม่ทราบค่า หรือเกิดการขับชนกันของสัญญาณ (Unknown / Contention)

- Z: ความต้านทานสูง หรือสภาวะเบี่ดวงจร (High-Impedance / Tristate)
- รูปแบบตัวเลข: <size>'<base><value> เช่น 8'hA5 = 8 บิต ฐาน 16 (เท่ากับ 8'b10100101)

4. ตัวดำเนินการ (Operators) (Slide 16-20)

- Bitwise Operators: คำนวณทีละบิต เช่น &, |, ^, ~ (เช่น 4'b1010 & 4'b1100 = 4'b1000)
- Logical Operators: ประเมินค่าความจริง ให้ผลลัพธ์ 1 บิต (1'b0 หรือ 1'b1) เช่น &&, ||, ! (เช่น 4'b1010 && 4'b1100 = 1'b1)
- Reduction Operators: กระทำกับทุกบิตภายในตัวแปรเดียวกัน ให้ผล 1 บิต เช่น ^4'b1101 = $1 \oplus 1 \oplus 0 \oplus 1 = 1\text{'b1}$ (ใช้ทำ Parity generator)

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 3

แหล่งอ้างอิง: [lab3.pdf](lab/lab3.pdf) (หน้าที่ 1-8)

1. เฉลยโค้ดและคำอธิบายทางเทคนิค

- Exercise 1: Basic Logic Gates

การเขียนเกตตรรกะ AND, OR, NOT, XOR ด้วยคำสั่ง assign:

verilog
```
assign and_out = a & b;
assign or_out  = a | b;
assign not_out = ~a;
assign xor_out = a ^ b;
```

- Exercise 2: Full Adder (Dataflow Modeling)

verilog
```
module full_adder(input a, b, cin, output sum, cout);
assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

- คำอธิบาย: sum เกิดจากการ XOR กันของอินพุตทั้งสามบิต ส่วน cout เป็น 1 เมื่อมีอินพุตอย่างน้อยสองตัวเป็น 1

- Exercise 3: วงจรเสียงข้างมาก (3-Input Majority Circuit)

verilog
```
module majority3(input a, b, c, output m);
assign m = (a & b) | (b & c) | (a & c);
endmodule
```

- คำอธิบาย: เอาต์พุต m จะเป็น 1 เมื่ออินพุตเกินครึ่ง (2 ใน 3 ขา) เป็น 1 สอดคล้องกับพจน์ $AB + BC + AC$

[WEEK] Week 4: Combinational Logic Design

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week4_Combinational_Design.pdf](lec/Week4_Combinational_Design.pdf) (Slide 1-26)

1. วัตถุประสงค์และภาพรวม (Slide 2-4)

- คุณสมบัติของ Combinational Logic: เอาต์พุตขึ้นกับอินพุตปัจจุบันเท่านั้น ไม่มีหน่วยความจำ และห้ามมี Feedback loop
- การเขียนวงจรด้วย Procedural block always @() ร่วมกับ case และ if-else
- การออกแบบ Multiplexer, Decoder, Priority Encoder, ALU และ 7-Segment Decoder
- กลไกการเกิดและวิธีป้องกัน Inferred Latch

2. ปัญหา Inferred Latch ในการออกแบบฮาร์ดแวร์ (Slide 21-23)

- สาเหตุ: เกิดขึ้นเมื่อตัวแปรในบล็อก always @() ไม่ได้รับค่าครบในทุกเงื่อนไขที่เป็นไปได้ เช่น if ขาด else หรือ case ขาดกรณีที่เป็นไปได้และไม่มี default:
- ผลกระทบ: ตัวแปรจะต้องคงค่าเดิมไว้ ทำให้ Quartus สังเคราะห์เป็น Level-Sensitive Latch ขึ้นมาโดยไม่ตั้งใจ วงจรจะเกิด Glitch และมีความเสี่ยงต่อ Timing Violation
- แนวทางแก้ไข (Golden Rule):

1. ประกาศค่าเริ่มต้น (Default Assignment) ไว้ที่บรรทัดบนสุดของบล็อก always @()
2. ใส่ else เสมอในทุกคำสั่ง if
3. ใส่ default: เสมอในทุกคำสั่ง case

3. วงจรสำคัญใน Lecture (Slide 8-20)

- Multiplexer: วงจรคัดเลือกข้อมูล 1 ช่องสัญญาณจากหลายช่องสัญญาณ ควบคุมด้วยขา sel
- Decoder: วงจรถอดรหัสไบนารีเป็น One-Hot เช่น 2-to-4 หรือ 3-to-8 Decoder
- Priority Encoder: ใช้งานคำสั่ง casez ร่วมกับเครื่องหมาย ? เพื่อระบุ Don't-Care บิต โดยให้ความสำคัญกับบิตที่มีนัยสำคัญสูงสุด (MSB) ก่อนเสมอ
- 7-Segment Display Decoder: บนบอร์ด DE0 เป็นแบบ Common Anode (Active-Low: ลอจิก 0 ติด, ลอจิก 1 ดับ)

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 4

แหล่งอ้างอิง: [lab 4.pdf](lab/lab%204.pdf) (หน้าที่ 1-10)

1. เฉลยโค้ดและคำอธิบายทางเทคนิค

- Exercise 1: 4-to-1 Multiplexer (mux4to1.v)

verilog
module mux4to1(input [3:0] in, input [1:0] sel, output reg y);
always @() begin
case (sel)
2'b00: y = in[0];
2'b01: y = in[1];
2'b10: y = in[2];
2'b11: y = in[3];
default: y = 1'b0; // ป้องกัน Latch
endcase
end
endmodule

- Exercise 4: BCD to 7-Segment Decoder (bcd_to_7seg.v)

กำหนดขาพอร์ต output reg [6:0] seg เรียงตาม {g, f, e, d, c, b, a}:
- ตัวเลข 0: ดับเฉพาะส่วน g $\rightarrow$ seg = 7'b1000000; (7'h40)
- ตัวเลข 8: ติดครบทุกส่วน $\rightarrow$ seg = 7'b0000000; (7'h00)
- ดับทุกส่วน: $\rightarrow$ seg = 7'b1111111; (7'h7F)

[WEEK] Week 5: Sequential Logic Design

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week5_sequential_logic.pdf](lec/Week5_sequential_logic.pdf) (Slide 1-25)

1. วัตถุประสงค์และภาพรวม (Slide 2-5)

- เข้าใจความแตกต่างระหว่าง Latch (Level-Sensitive) กับ Flip-Flop (Edge-Triggered)
- โครงสร้าง D Flip-Flop, Asynchronous Reset และ Synchronous Reset
- กฎเหล็กของการกำหนดค่า (Assignment Rules): การใช้ Blocking (=) และ Non-blocking (<=)
- กลไกการป้องกัน Race Condition ในวงจรตามลำดับ

2. Asynchronous Reset vs Synchronous Reset (Slide 9-10)

- Active-Low Asynchronous Reset:

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n) q <= 1'b0; // ทำงานทันทีที rst_n เป็น 0 ไม่ต้องรอ clk
else q <= d;
end

สัญญาณ rst_n อยู่ใน Sensitivity List มีลำดับความสำคัญสูงสุด ใช้สำหรับรีเซ็ตระบบทันทีตอนเปิดเครื่อง

- Synchronous Reset:

verilog
always @(posedge clk) begin
if (!rst_n) q <= 1'b0; // ทำงานทีขอบขาขึนของ clk เท่านัน
else q <= d;
end

สัญญาณ rst_n ไม่อยู่ใน Sensitivity List ปลอดภัยจากสัญญาณรบกวน (Glitch) บนสายรีเซ็ต

3. กฎทองคำของ Blocking (`=`) และ Non-blocking (`<=`) (Slide 14-18)

1. ใน Sequential Always Block (@(posedge clk)): ให้ใช้ Non-blocking Assignment (<=) เสมอ

- เหตุผล: สัญญาณ Non-blocking ทังหมดใน time-step เดียวกันจะถูกประเมินค่าฝังขวามือพร้อมกัน แล้วอัปเดตลงตัวแปรฝังซ้ายมือพร้อมกันในคิวเฟส NBA (Non-Blocking Assignment) ของ Simulator ทำให้จำลองพฤติกรรมการทำงานขนานของ Flip-Flop จริงได้อย่างถูกต้อง

2. ใน Combinational Always Block (@()): ให้ใช้ Blocking Assignment (=) เสมอ

3. ข้อห้ามเด็ดขาด (Illegal):

- ห้ามใช้ = และ <= ปะปนกันภายในบล็อก always เดียวกัน

- ห้ามกำหนดค่าตัวแปร reg ตัวเดียวกันจากหลายๆ บล็อก always (เกิดปัญหา Multiple Drivers)

[LAB] ส่วนที 2: สรุปและเฉลยใบงาน LAB 5

แหล่งอ้างอิง: [lab 5.pdf](lab/lab%205.pdf) (หน้าที 1-8)

1. เฉลยโค้ดและคำอธิบายทางเทคนิค

- Exercise 3: 4-bit Register with Load Enable

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
q <= 4'd0;
else if (load)
q <= d;
// เมือ load == 0 วงจรจะคงค่าเดิมไว้ (Implicit Hold: q <= q;)
end

- Exercise 4: 4-bit Shift Register (SIPO)

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
pout <= 4'b0000;
else
pout <= {pout[2:0], sin}; // เลือนบิตเดิมไปทางซ้าย รับ sin เข้าทางขวา
end

[WEEK] Week 6: Counters & Shift Registers

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week6_Counters_ShiftRegisters.pdf](lec/Week6_Counters_ShiftRegisters.pdf) (Slide 1-25)

1. วัตถุประสงค์และภาพรวม (Slide 2-5)

- ออกแบบวงจรนับแบบซิงโครนัส (Synchronous Counter) และวงจรนับแบบขึ้น/ลง (Up/Down Counter)
- คำนวณวงจรหารความถี่ (Clock Divider) และการใช้เทคนิค Synchronous Clock Enable
- ออกแบบวงจรเลื่อนข้อมูล (Shift Registers): SIPO, PISO, PIPO, SISO
- เปรียบเทียบ Ring Counter กับ Johnson Counter

2. การคำนวณวงจรหารความถี่ (Clock Divider) (Slide 11-13)

- สัญญาณนาฬิกาหลักบนบอร์ด DE0: $f_{\text{in}} = 50\text{ MHz} = 50,000,000\text{ Hz}$
- ต้องการสร้างพัลส์ความถี่ $f_{\text{out}} = 1\text{ Hz}$ (คาบเวลา 1 วินาที):

$$\text{จำนวนรอบที่ต้องนับ} = \frac{f_{\text{in}}}{f_{\text{out}}} = \frac{50,000,000}{1} = 50,000,000\text{ รอบ}$$

- เนื่องจากตัวนับเริ่มจาก 0 ค่าสูงสุดที่ต้องเทียบคือ:

$$\text{Terminal Count} = 50,000,000 - 1 = 49,999,999$$

- จำนวนบิตของ Register ที่ต้องใช้:

$$B = \lceil \log_2 50,000,000 \rceil = 26\text{ บิต} \quad (\text{เนื่องจาก } 2^{25} = 33,554,432 < 50,000,000 < 2^{26} = 67,108,864)$$

3. Ring Counter vs Johnson Counter (Slide 20)

- Ring Counter: ขนาด N บิต มีจำนวนสถานะ N สถานะ โดยวนบิต 1 เพียงบิตเดียว
- Johnson Counter: ขนาด N บิต มีจำนวนสถานะ $2N$ สถานะ โดยนำเอาต์พุตตัวสุดท้ายกลับค่า (Inverted feedback) วนกลับมายังตัวแรก

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 6

แหล่งอ้างอิง: [Lab6.pdf](lab/Lab6.pdf) (หน้าที่ 1-8)

1. เฉลยโค้ดและคำอธิบายทางเทคนิค

- Section 2: Mod-10 Up/Down Counter (mod10_counter.v)

```
verilog
if (updown) begin
if (count == 4'd9) count <= 4'd0; // นับขึ้นถึง 9 แล้ววนกลับ 0
else               count <= count + 4'd1;
end else begin
if (count == 4'd0) count <= 4'd9; // นับลงถึง 0 แล้ววนกลับ 9
else               count <= count - 4'd1;
end
```

- Section 4: Running LED Controller (running_led.v)

เพื่อความเสถียรของวงจร ห้ามนำสัญญาณ 1 Hz ไปต่อเป็น Clock ของ Flip-Flop โดยตรง (เป็นการสร้าง Asynchronous Clock Domain) ให้ใช้ CLOCK_50 เป็นสัญญาณนาฬิกาหลัก แล้วนำพัลส์ 1 Hz มาเป็นสัญญาณ Enable:

```
verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
led <= 8'b00000001;
else if (tick_1hz && !pause) begin
if (dir) led <= {led[6:0], led[7]}; // หมุนซ้าย
else     led <= {led[0], led[7:1]}; // หมุนขวา
end
```

end

[WEEK] Week 7: Finite State Machine (FSM) Design

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week7_FSM.pdf](lec/Week7_FSM.pdf) (Slide 1-33)

1. วัตถุประสงค์และองค์ประกอบ 4 ส่วนของ FSM (Slide 3-4)

FSM ประกอบด้วย 4 องค์ประกอบสำคัญ:

1. State Register (Sequential): เก็บสถานะปัจจุบัน (current_state)
2. Next-State Logic (Combinational): คำนวณสถานะถัดไป (next_state) จาก state และ input
3. Output Logic (Combinational): สร้างเอาต์พุตตามประเภท FSM
4. Inputs / Outputs: สัญญาณควบคุมและสัญญาณเอาต์พุต

2. การเปรียบเทียบ Moore FSM กับ Mealy FSM (Slide 4-6)

คุณสมบัติ	Moore FSM	Mealy FSM
Output Dependency	ขึนกับ Current State เท่านัน	ขึนกับ Current State และ Current I
ความเสียงต่อ Glitch	ปลอดภัย ไม่มี Glitch (Synchronous)	เสียงเกิด Glitch (มี combinational
ความเร็วตอบสนอง	ช้ากว่า 1 clock cycle	ตอบสนองได้ทันทีใน cycle เดียวกัน
จำนวนสถานะ	มักใช้จำนวนสถานะมากกว่า	มักใช้จำนวนสถานะน้อยกว่าหรือเท่ากัน
ใน State Diagram	เขียน Output ในวงกลม State	เขียน Output บนลูกศร Transition

3. รูปแบบ State Encoding (Slide 7)

- Binary Encoding: ใช้ Flip-Flop ขันต่ำ $\lceil \log_2 N \rceil$ ตัว (เช่น 6 สถานะ ใช้ 3 ตัว)
- One-Hot Encoding: ใช้ Flip-Flop 1 ตัวต่อ 1 สถานะ (N ตัวสำหรับ N สถานะ) เป็นทีนิยมมากบน FPGA เพราะลดความซับซ้อนของ Next-State Logic และทำงานได้ความเร็วสูงกว่า

4. โครงสร้าง Moore 3-Always-Block Style (Slide 11)

```
// Block 1: State Register (Sequential)
always @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE; else state <= next_state;

// Block 2: Next-State Logic (Combinational)
always @(*) begin
  next_state = state;
  case (state)
    IDLE: if (in) next_state = S1;
    S1:   if (in) next_state = S2; else next_state = IDLE;
    default: next_state = IDLE;
  endcase
end

// Block 3: Output Logic (Combinational - Moore)
always @(*) begin
  out = 1'b0;
  case (state)
    IDLE: out = 1'b0;
    S2:   out = 1'b1;
    default: out = 1'b0;
  endcase
end
```

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 7

แหล่งอ้างอิง: [Lab7.pdf](lab/Lab7.pdf) (หน้าที่ 1-12)

1. เฉลยระบบ Combination Lock System

- รหัสผ่านเริ่มต้น: PW0 = 3, PW1 = 7, PW2 = 1
- ลำดับสถานะ (Moore FSM):

1. IDLE $\rightarrow$ กดถูกไป D1_OK, กดผิดไป WRONG
2. D1_OK $\rightarrow$ กดถูกไป D2_OK, กดผิดไป WRONG
3. D2_OK $\rightarrow$ กดถูกไป UNLOCKED, กดผิดไป WRONG
4. UNLOCKED $\rightarrow$ เปิดประตู (unlocked=1), หน่วงเวลา 5 วินาทีแล้ว Auto-lock กลับสู่ IDLE

5. WRONG $\rightarrow$ เพิ่ม attempt_cnt หน่วงเวลา 1 วินาที หากสะสมครบ 3 ครั้งจะเข้าสู่ LOCKOUT มิฉะนั้นกลับไป IDLE ให้ลองใหม่

6. LOCKOUT $\rightarrow$ ร้องเตือนภัย (alarm = 10'h3FF LEDR ติดทุกดวง) และนับถอยหลัง $10 \rightarrow 0$ บน HEX0 ก่อนปลดล็อกกลับไป IDLE

2. วงจรตรวจจับขอบขาลง (Edge Detector ใน `edge_detector.v`)

```
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin r1 <= 1'b1; r2 <= 1'b1; end
  else        begin r1 <= key_in; r2 <= r1; end
end
assign pulse_out = r2 & ~r1; // ▯▯▯▯▯▯▯▯ Falling Edge ▯▯▯▯▯▯▯▯ Active-Low
```

[WEEK] Week 8: Testbench Design & Functional Verification

[LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

แหล่งอ้างอิง: [Week8.pdf](lec/Week8.pdf) (Slide 1-33)

1. วัตถุประสงค์และโครงสร้าง Testbench 4 ส่วน (Slide 3-6)

Testbench ไม่สามารถสังเคราะห์เป็นฮาร์ดแวร์ได้ (Non-synthesizable) มี 4 องค์ประกอบ:

1. Clock & Reset Generator: สร้างสัญญาณฐานเวลา เช่น always #5 clk = ~clk;
2. Stimulus Generator: สร้างชุดสัญญาณอินพุตทดสอบใน initial begin ... end
3. DUT (Device Under Test): โมดูลที่เราออกแบบ เชื่อมต่อแบบ Named Port Mapping
4. Checker / Scoreboard: ตรวจสอบค่าจริงเทียบกับค่าคาดหวัง และรายงาน PASS/FAIL

2. หัวใจของ Self-Checking Testbench (Slide 8-10)

- โปรแกรมตัดสินผล PASS/FAIL อัตโนมัติด้วยค่าสังเปรียบเทียบและ $display ไม่ต้องนั่งดู Waveform ด้วยตา

- การใช้ Case Equality (=== และ !==): ตรวจจับสถานะ X (Unknown) และ Z (High-Z) ได้แม่นยำ แตกต่างจาก == และ != ที่จะถือเป็นเท็จในเงื่อนไข if

- การหน่วงเวลา #1; เพื่อป้องกัน Race Condition:

verilog

rst=1; en=0; @(posedge clk); #1; // หน่วงเวลา #1 รอให้ non-blocking ใน DUT settle

check_count(4'd0, "after reset");

หากไม่มี #1; โปรแกรมจะอ่านค่าเอาต์พุตใน Active region ณ ขอบขา clock ซึ่งจะได้ค่าเก่าก่อนอัปเดต ทำให้รายงาน FAIL ผิดพลาด

3. ระดับการจำลองการทำงาน (Simulation Levels) (Slide 19-22)

1. RTL Simulation (Functional): แบบ Zero-delay รันเร็วที่สุด ตรวจสอบตรรกะเบื้องต้น ไม่เห็นปัญหา Glitch และ Timing

2. Post-Synthesis Simulation: จำลองจาก Gate Netlist ตรวจจับ Inferred Latch และ Dead logic

3. Post-Implementation (Timing) Simulation: รวม Gate delay และ Wire routing delay จากชิปจริงผ่านไฟล์ SDF ใกล้เคียงฮาร์ดแวร์จริงที่สุด ตรวจพบ Glitch และ Setup/Hold violation

4. System Tasks และ Coverage (Slide 11-13, 29)

- $display: พิมพ์ทันทีที่เรียก (Active region)
- $monitor: พิมพ์อัตโนมัติเมื่อตัวแปรเปลี่ยนค่า

- $strobe: พิมพ์ที่ท้ายสุดของ time step (Postponed region) หลัง non-blocking ทุกตัว settle แล้ว

- Functional Coverage: ตรวจสอบว่าเงื่อนไขตาม Specification (เช่น ทุก State และทุก Transition ใน FSM) ถูกทดสอบครบหรือไม่ (ไม่เหมือน Code Coverage ที่ดูแค่การรันบรรทัดโค้ด)

[LAB] ส่วนที่ 2: สรุปและเฉลยใบงาน LAB 8

แหล่งอ้างอิง: [Lab8.pdf](lab/Lab8.pdf) (หน้าที่ 1-16)

1. เฉลยแบบฝึกหัด Debugging Case Studies

- Case Study 1: Shift Register TB (หน้า 4)

- บัก 1 (บรรทัด 11): ใช้ = แทน === (เป็น assignment) $\rightarrow$ แก้เป็น if (pout === exp)

- บัก 2 (บรรทัด 21, 23): ขาด #1; หลัง @(posedge clk) $\rightarrow$ ใส่ #1; ก่อนเรียก check_pout

- บัก 3 (บรรทัด 29): ลืมใส่ $finish; $\rightarrow$ เพิ่ม $finish; ท้าย initial block

- Case Study 2: Priority Encoder TB (หน้า 5)

- บัก 1 (บรรทัด 11): ใช้ = ใน out = exp_out $\rightarrow$ แก้เป็น out === exp_out && valid === exp_valid

- บัก 2 (บรรทัด 11-16): ตรรกะ if-else สลับทิศ เมื่อค่าตรงกันกลับรายงาน [FAIL] $\rightarrow$ สลับบล็อกโค้ดของ PASS และ FAIL

2. เฉลยช่องว่าง Self-Checking TB (Mod-10 Counter & Traffic Light)

- Mod-10 Counter TB (หน้า 12):

(1) ~clk; | (2) !== | (3) #1; | (4) repeat | (5) 4'd9 | (6) $finish;

- Traffic Light FSM TB (หน้า 15-16):

(1) 1'b0; | (2) 100 | (3) 4 | (4) 010 | (5) 2 | (6) $finish;

[ตาราง] ตารางสรุปภาพรวมความเชื่อมโยงตลอดหลักสูตร (Week 2 - Week 8)

สัปดาห์	หัวข้อ LECTURE (สไลด์บรรยาย)	หัวข้อ LAB (ใบงานปฏิบัติการ)	แนวคิดหลักที่เชื่อมโยงกันอย่างเป็นร
Week 2	Quartus II Installation & Design Fl	Practice Lab Sheet 2: Project Setup	การแปลงแนวคิดสู่วงจรฮาร์ดแวร์จริงผ่
Week 3	Foundations & Language Basics	Lab 3: Basic Gates & Structural/Dat	โครงสร้างโมดูล, Named vs Positional
Week 4	Combinational Logic Design	Lab 4: Multiplexer, Decoder, ALU &	วงจรไร้หน่วยความจำ, `case`/`if-else
Week 5	Sequential Logic Design	Lab 5: Latches, Flip-Flops & Clocke	วงจรหน่วยความจำตามสัญญาณนาฬิกา, Asy
Week 6	Counters and Shift Registers	Lab 6: Mod-N Counters, Clock Divide	วงจรนับเชิงเวลา: การหารความถี่ 50 M
Week 7	Finite State Machine (FSM) Design	Lab 7: Combination Lock System (Moo	การควบคุมเชิงลำดับ: สถาปัตยกรรม Moo
Week 8	Testbench Design & Verification	Lab 8: Self-Checking TB for Counter	การตรวจสอบอัตโนมัติ (Simulation-onl

---

[WEEK] Lab 9 (Special): การออกแบบวงจร AES-128 Encryption ด้วย Generative AI

แหล่งอ้างอิง: [lab9_Lab_AES128_Encryption 1.pdf](file:///d:/vhdl/lab/lab9_Lab_AES128_Encryption%201.pdf) (หน้าที่ 1-12)

1. วัตถุประสงค์และสเปกมาตรฐาน (หน้าที่ 1-3)

- วงจรรับ Plaintext ขนาด 128 บิต (16 ไบต์) และ Cipher Key ขนาด 128 บิต เข้าคำนวณ Ciphertext ขนาด 128 บิต ตามมาตรฐาน NIST FIPS-197

- วงจรทำหน้าที่เข้ารหัส (Encryption) เพียงอย่างเดียว ทำงานแบบ Synchronous Sequential

2. ลำดับขั้นตอนการทำงาน 10 Rounds ของ AES-128 (หน้าที่ 3)

1. Round 0: state = plaintext XOR round_key[0] (ทำเฉพาะ AddRoundKey เท่านั้น)

2. Round 1 ถึง 9: ทำครบ 4 ขั้นตอน:

- SubBytes: การแทนที่ไบต์แบบ Non-linear ผ่านตาราง S-Box

- ShiftRows: การเลื่อนไบต์ในแต่ละแถวของ State Matrix

- MixColumns: การคูณเชิงเมทริกซ์ในฟิลด์จำกัด Galois Field GF(2^8) เพื่อกระจายบิต (Diffusion)

- AddRoundKey: นำ State มา XOR กับกุญแจย่อยประจำรอบ round_key[r]

3. Round 10 (รอบสุดท้าย):

- state = AddRoundKey(ShiftRows(SubBytes(state)), round_key[10])

- ข้อควรระวังสำคัญ: ใน Round 10 จะไม่มีขั้นตอน MixColumns โดยเด็ดขาดตามมาตรฐานสากล

3. หลักการคณิตศาสตร์ของ S-Box (หน้าที 4)

- S-Box ไม่ใช่การสุ่มค่า แต่สร้างขึนจาก GF(2^8) Multiplicative Inverse ร่วมกับ Affine Transformation เหนือ $GF(2)$ เพื่อสร้างคุณสมบัติ Confusion ทีสูงสุด

[WEEK] Lab 11 (Special): ตัวนับขึน/ลงบน 7-Segment ด้วย RV32I SoC

แหล่งอ้างอิง: [Lab11_SoC_7seg_Counter.pdf](file:///d:/vhdl/lab/Lab11_SoC_7seg_Counter.pdf) (หน้าที 1-4)

1. โครงสร้างหน่วยประมวลผลและการจัดสรรหน่วยความจำ (Memory Map) (หน้าที 2)

- ระบบใช้ซอฟต์คอร์ซีพียู RISC-V สถาปัตยกรรม RV32I Single-Cycle
- แผนทีหน่วยความจำ (Address Decoder addr_decoder.v จากบิต addr[31:28]):

- 0x0000_0000: Instruction ROM (imem.v) โหลดจากไฟล์ไบนารี main.hex

- 0x1000_0000: Data RAM (dmem.v) สำหรับตัวแปรและ Stack

- 0x2000_0000: Memory-Mapped I/O Registers สำหรับควบคุมจอ 7-Segment และปุ่มกด

2. การแบ่งหน้าทีระหว่างฮาร์ดแวร์และซอฟต์แวร์ (Hardware/Software Co-Design) (หน้าที 3-4)

- ฮาร์ดแวร์ (hex7seg.v): ทำหน้าทีถอดรหัสเลขฐานสอง 4 บิต (0-F) ออกเป็นสัญญาณดิดดับของแต่ละขีด 7 ส่วน (Common Anode Active-Low)

- ซอฟต์แวร์ (ภาษา C บน RV32I): โปรแกรมภาษา C มีหน้าทีเพียงเขียนค่าตัวเลขลงใน Memory-Mapped Register เช่น HEX0 = count; โดยไม่ต้องถอดรหัสบิตของ 7-segment เองในโค้ด C

- การตรวจจับปุ่มกดในระดับซอฟต์แวร์ (Software Edge Detection):

- ความเร็ว Loop ของซีพียูเร็วกว่าการกดปุ่มของมนุษย์มาก

- ในภาษา C ต้องเก็บค่าปุ่มของรอบก่อนหน้า (KEY_prev) เพื่อนำมาเปรียบเทียบกับรอบปัจจุบัน (KEY_curr)
โดยนับเฉพาะจังหวะทีปุ่มเปลียนสถานะจากไม่กดเป็นกดเท่านัน เพื่อป้องกันการนับซำหลายรอบในการกดปุ่มค้าง